

SELENIUM

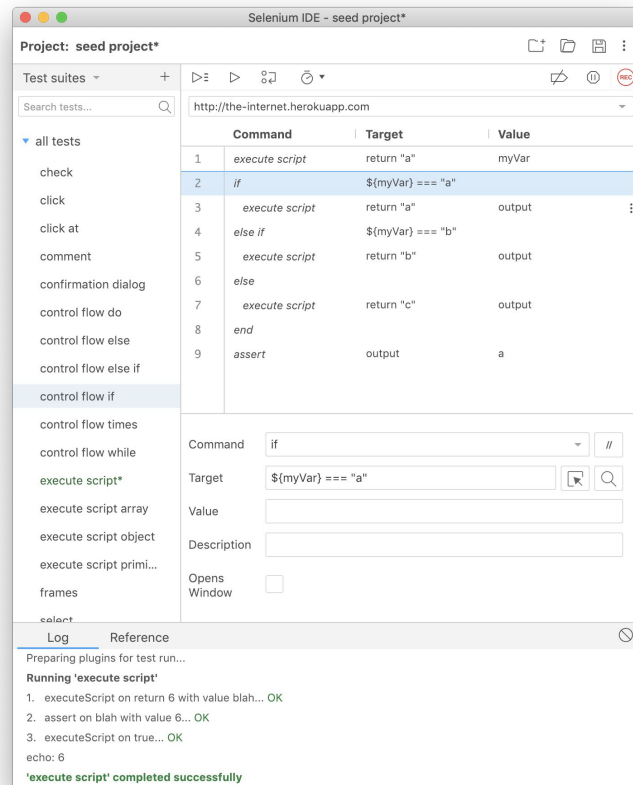
Selenium

- selenium.dev.
- *Framework* para automação de navegadores.
- Não é um *framework* de teste.
- Utilizado em conjunto com *frameworks* de teste.
- Realização de testes de aceitação para aplicações *Web*.
- Selenium IDE e Selenium WebDriver.

Selenium IDE

- Permite gravar, salvar, editar e reproduzir interações com o navegador.
- Extensão para o Chrome e Firefox.
- Usado pelos desenvolvedores do teste para obter o caminho dos elementos.
- Usado para automatizar tarefas no navegador.
- Utiliza internamente o Selenium WebDriver.

Selenium IDE



Selenium WebDriver

- Permite controlar navegadores programaticamente.
- Possui bibliotecas específicas para cada linguagem.
- Suporte a diferentes navegadores.
- Se comunica com os navegadores através de *drivers*.
- Suporte a navegadores virtuais como HtmlUnit e PhantomJS.
- Selenium-RC e versões antigas.

Erros comuns

- `java.lang.IllegalStateException`: The path to the driver executable must be set by the `webdriver.chrome.driver` system property
- `java.lang.IllegalStateException`: The driver is not executable
- `org.openqa.selenium.SessionNotCreatedException`: session not created: This version of ChromeDriver only supports Chrome version 86. Current browser version is 85.0.4183.102

Problemas com páginas dinâmicas e assíncronas

- A execução do selenium é normalmente síncrona.
- Após executar um comando, o próximo será executado em seguida.
- Entretanto, a aplicação pode demorar para executar algumas operações.
- O elemento do próximo comando pode ainda não estar na página.
- O elemento do próximo comando pode ainda não estar pronto.

Resolver problemas de sincronismo

- Utilizar `Thread.sleep` entre um comando outro.
- Não é o método mais adequado.
- O Selenium suporta três tipos adequados de espera.
- Espera implícita.
- Espera explícita.
- Espera fluente.

Espera implícita

- Configurado apenas uma vez e utilizado implicitamente no `findElement`.
- Selenium espera o máximo de um timeout até que o elemento apareça.
- Se o elemento não aparecer até o *timeout*, então uma exceção é lançada.

Espera implícita

```
seleniumImplicito = new ChromeDriver();  
seleniumImplicito.manage().timeouts()  
    .implicitlyWait(5, TimeUnit.SECONDS);  
  
seleniumImplicito.findElement(By.id("botaoCadastrar"));
```

Espera explícita

- Depende de um *timeout* e de uma condição.
- Selenium espera o máximo do *timeout* até que a condição seja satisfeita.
- Se a condição não for satisfeita até o *timeout*, então uma exceção é lançada.

Espera explícita

```
seleniumImplicito = new ChromeDriver();  
seleniumExplicito = new WebDriverWait(seleniumImplicito,  
    Duration.ofSeconds(5));  
  
seleniumExplicito.until(ExpectedConditions  
    .elementToBeClickable(By.id("botaoEnviar")));
```

Espera fluente

- Depende de um *timeout*, de um intervalo entre tentativas e de um predicado.
- Selenium espera o máximo do *timeout* até que o predicado seja satisfeito.
- Selenium fará uma tentativa a cada intervalo.
- Se o predicado não for satisfeito até o *timeout*, uma exceção é lançada.

Espera explícita

```
seleniumImplicito = new ChromeDriver();
seleniumFluente = new FluentWait<>(seleniumImplicito);
seleniumFluente.withTimeout(Duration.ofSeconds(10));
seleniumFluente.pollingEvery(Duration.ofSeconds(2));
seleniumFluente.ignoring(NoSuchElementException.class);
seleniumFluente.until(driver -> {
    String elemento = driver.findElement(By.id("mensagens"));
    String mensagem = elemento.getText();
    return mensagem.equals("Cadastro com sucesso!");
});
```